LibSpace

Project Title

LibSpace API – A Robust Backend for Library Operation

1. Project Overview

LibSpace is a micro web-based library management system that provides secure user authentication and basic book management features.

The application allows users to:

- Register a new account
- Log in using credentials
- Log out securely
- Delete their accounts

The main goal of LibSpace is to demonstrate authentication flow, CRUD operations, backend development, and database integration in a structured manner.

2. Project Instructions – LibSpace

The following steps describe how users interact with the system:

1. Register

Register a new user account using the registration form.

2. Login

Log in using the registered credentials to access the dashboard.

3. Book Management

- Add a new book
- View the list of available books
- Get book by ID
- Update book details
- Delete a book

4. Logout

Users can securely log out after completing operations.

5. Delete Account

Users also have the option to delete their account permanently from the system.

Pre-Requisites

Before running or developing LibSpace, ensure the following:

Basic Knowledge

- JavaScript
- Backend concepts (Node.js / Express)
- Database fundamentals (CRUD operations)

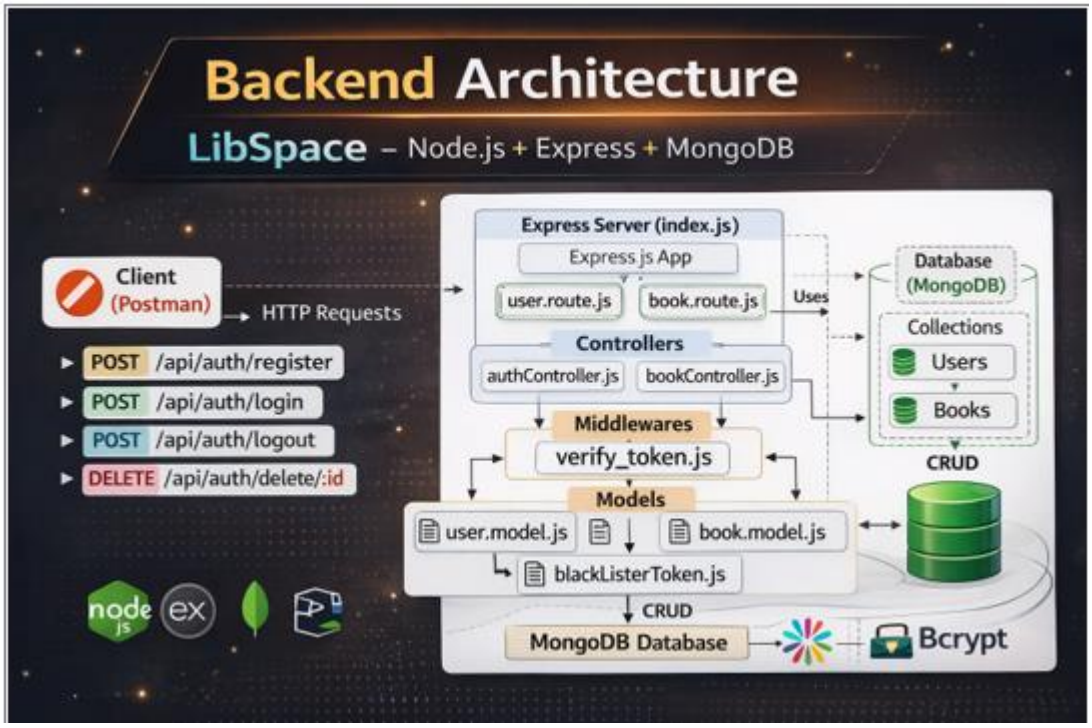
Software Requirements

- Node.js and npm installed
- Database setup (MongoDB / MySQL)
- Code editor (VS Code)
- Web browser (Chrome / Firefox)

Concept Knowledge

- REST APIs
- Authentication (JWT / Sessions)
- Environment variables configuration

3. Project Architecture



Technical Architecture

LibSpace follows a Layered Backend Architecture consisting of:

1. API Layer (Routes)
2. Business Logic Layer (Controllers)
3. Data Access Layer (Models / Database)

This structure ensures clean code organization, scalability, and maintainability.

1. API Layer (Routes)

The API layer handles incoming HTTP requests and defines application endpoints.

- Built using:
  - Node.js
  - Express.js
- Responsibilities include:
  - Defining routes (e.g., /register, /login, /books)
  - Mapping routes to controllers
  - Handling HTTP methods (GET, POST, PUT, DELETE)
  - Applying middleware (authentication, validation)

Example Endpoints

- POST → Register User
- POST → Login User
- DELETE → Delete User
- POST → Add Book
- GET → Fetch Books
- PUT → Update Book
- DELETE → Delete Book

2. Business Logic Layer (Controllers)

The controller layer contains the core application logic.

- Responsibilities include:
  - Processing user registration and login
  - Password hashing using bcrypt
  - Token generation using JWT
  - Implementing CRUD operations for books

- Validating request data
- Handling errors and sending responses

This layer acts as a bridge between routes and the database.

User Controller

File: controller/user.controller.js

```
import User from "../models/user.model.js";
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
import BlackList from "../models/blackListerToken.js";


const userController = {
  registerUser: async (req, res) => {
    const { name, email, password } = req.body;
    try {
      if (!name || !email || !password) {
        return res.json({
          success: false,
          message: "All details are required...",
        });
      }

      const checkExists = await User.find({ email: email });
      console.log(checkExists);
      if (checkExists.length !== 0) {
        return res.json({
          success: false,
          message: "Account already exists, please login",
        });
      }
      const hashPassword = await bcrypt.hash(password, 10);
      const newUser = new User({
        name,
        email,
        password: hashPassword,
      });
      newUser.save();
```

```
      newUser.save();

      return res.json({
        success: true,
        message: "User registered successfully!",
      });
    } catch (error) {
      return res.json({
        success: false,
        error: error.message,
      });
    }
  },
  loginUser: async (req, res) => {
    const { email, password } = req.body;

    console.log("Email", email, "Password", password);
    try {
      if (!email || !password) {
        return res.json({
          success: false,
          message: "All fields are required.",
        });
      }
      const checkExists = await User.find({ email: email });
      console.log(checkExists);
      if (checkExists.length == 0) {
        return res.json({
          success: false,
          message: "Account , doesn't exists.",
        });
      }

      await bcrypt.compare(password, checkExists[0].password, (err, result) => {
        console.log(result);
        if (result) {
          const token = jwt.sign(
            { id: checkExists[0]._id, email: checkExists[0].email },
            process.env.SECRET_KEY,
            {
              expiresIn: "1h",
            },
          );
          return res.status(200).send({
            msg: "Logged In Successfully",
            token: token,
            user: checkExists,
          });
        } else {
          console.error("Error", err);
          return res.status(404).send({ err: err });
        }
      });
    } catch (error) {
      return res.status(400).send({ error: error });
    }
  },
```

```
deleteUser: async (req, res) => {
  try {
    const { userId } = req.params;
    let findUser = await User.findById({ _id: userId });

    if (!findUser) {
      return res.status(404).send({ msg: "user not found" });
    }

    await User.findByIdAndDelete({ _id: userId });
    res.status(200).send({ msg: "User deleted successfully" });
  } catch (error) {
    res.status(403).json({
      status: false,
      message: error.message,
    });
  }
},
logoutUser: async (req, res) => {
  let token = req.headers.authorization.split(" ")[1];
  let blackList = await BlackList.findOne({ token: token });
  if (blackList) {
    return res
      .status(401)
      .send({ msg: "User has already logged out, please login" });
  } else {
    res.status(200).send({ msg: "User has logged out successfully" });
    let blackListedToken = new BlackList({
      token: token,
```

```
      });
      await blackListedToken.save();
    }
  },
};

export default userController;
```

Book Controller

File: controller/book.controller.js

The Book Controller contains the business logic for managing books in the LibSpace application.
It handles operations such as adding a book, retrieving books, updating book details, and deleting books.
Each book is associated with a specific user through a MongoDB ObjectId reference.

```
import Book from "../models/book.model.js";

const bookController = {
  addBook: async (req, res) => {
    const { url, title, author } = req.body;

    if (!url || !title || !author) {
      return res.status(400).send({
        msg: "All fields are required",
      });
    }
    console.log(req.user);
    const { id } = req.user;
    try {
      const checkIsAval = await Book.findOne({ title: title });
      if (checkIsAval) {
        return res.status(200).send({ msg: "Books   already exists" });
      }
      const newBook = new Book({
        url,
        user: id,
        title,
        author,
      });
      await newBook.save();
      res.send({ msg: "Book added successfully" });
    } catch (error) {
      res.status(403).send({ err: error.message });
    }
  },
  getBookById: async (req, res) => {
    const { bookId } = req.params;
```

```
    const { id } = req.user;
    try {
      const singleBook = await Book.find({ _id: bookId, user: id });
      if (!singleBook) {
        return res.status(404).send({ msg: "Please enter a valid Id" });
      }
      res.status(200).json(singleBook);
    } catch (error) {
      res.status(400).send({ err: error.message });
    }
  },
  getAllBooks: async (req, res) => {
    const { id } = req.user;
    try {
      const allBooks = await Book.find({ user: id });
      if (!allBooks) {
        return res.status(200).send({ msg: "Empty" });
      }
      return res.status(200).json(allBooks);
    } catch (error) {
      res.status(400).send({ err: error.message });
    }
  },
  updateBook: async (req, res) => {
  try {
    const bookId = req.params.bookId;

    const book = await Book.findOne({
      _id: bookId,
```

```
    user: req.user.id
  });

  if (!book) {
    return res.status(404).send({ msg: "Book not found" });
  }

  await Book.findByIdAndUpdate(bookId, {
    $set: req.body
  });

  res.send("Data updated successfully");

} catch (error) {
  res.status(400).send({ Error: error.message });
}
,

deleteBook: async (req, res) => {
  const { id } = req.user;
  try {
    let { bookId } = req.params;
    let book = await Book.findById({ _id: bookId, user: id });
    if (!book) {
      return res.status(404).send({ msg: "Book not found" });
    }

    await Book.findByIdAndDelete({ _id: bookId });
```

```
    res.status(200).send({ msg: "Book deleted successfully" });
  } catch (error) {
    res.send({ err: error.message });
  }
 },
};

export default bookController;
```

3. Data Access Layer (Models / Database)

The data layer manages persistent data storage.

- Database used:

  MongoDB

- Models define schema structures for:

Users

- username
- email
- password

Books

- title
- author
- published_date
- user reference

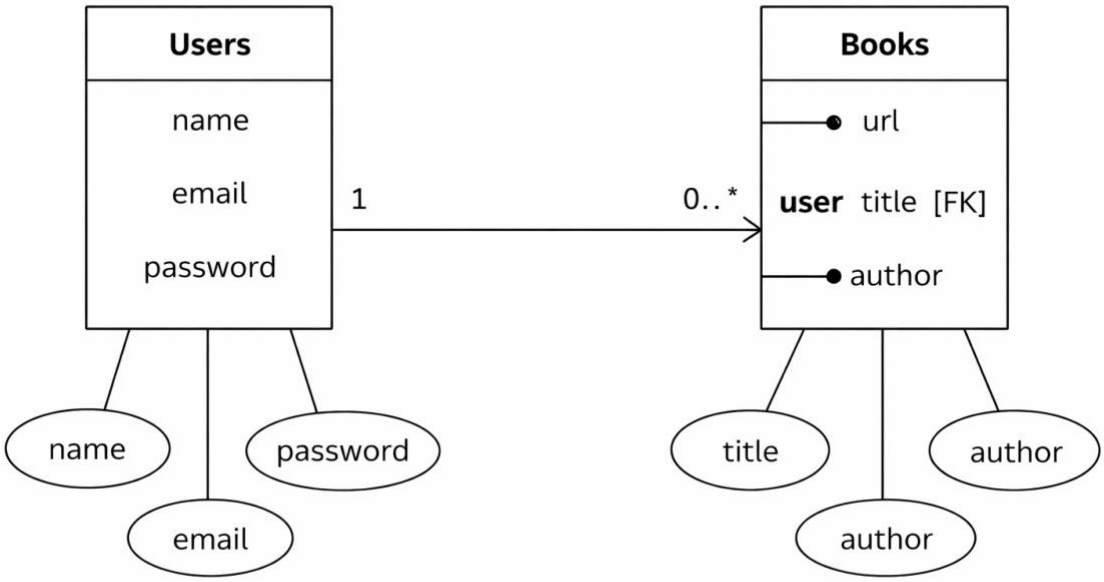
Responsibilities:

- Executing database queries
- Storing and retrieving data
- Maintaining data integrity

4. Request Flow

1. Client sends an HTTP request to the API endpoint.
2. The route receives the request and forwards it to the controller.
3. The controller processes the business logic.
4. The controller interacts with the database via models.
5. The database returns the data to the controller.
6. The controller sends a structured JSON response back to the client.

ER Diagram:



Features

1.User Authentication

- User Registration (name, email, password)
- Secure password storage using hashed passwords
- User login with credential verification
- Token-based authentication (JWT)
- Delete user account
- Automatic timestamps (createdAt, updatedAt)

2. Book Management (CRUD Operations)

- Each book is linked to a specific user using a reference (user: ObjectId).
- Users can:
- Add a new book (url, title, author)
- Get all books of a specific user
- Get book by ID
- Update book details
- Delete a book
- Books are accessible only to authenticated users.

3. User–Book Relationship

- The system follows a One-to-Many Relationship.
- One User → Can have multiple Books
- Each Book → Belongs to one User (via ObjectId reference)
- This maintains data integrity using MongoDB references.

4. Security & Validation

- Security features include:
- Input validation for required fields
- Protected routes using authentication middleware
- Error handling for invalid requests
- Unauthorized access prevention.

5. RESTful API Design

- The API follows REST principles.
- Standard HTTP methods used:
- POST
- GET
- PUT
- DELETE
- All responses return structured JSON data.

Roles and Responsibilities:

Requirement Analysis

- Understanding project requirements (authentication + book CRUD)
- Designing database schema
- Defining the relationship between user and books

Database Design

- Creating Mongoose schemas
- Designing user and book collections
- Establishing ObjectId references
- Ensuring data consistency

API Development

- Developing RESTful API endpoints including:
- User Registration
- User Login
- Delete User
- Add Book
- Get Books
- Update Book
- Delete Book
- Routes and controllers are structured to maintain modularity and scalability.

Authentication & Security

- Security measures implemented:
- Password hashing using bcrypt
- JWT token generation
- Middleware for route protection
- Validation for unauthorized access

Testing & Debugging

- APIs are tested using Postman.
- Testing includes:
- Authentication verification
- CRUD operations
- Error handling
- Correct HTTP response codes

User Flow – LibSpace Backend

1. User Registration

- POST /api/auth/register
- Steps:
- User submits name, email, password
- System validates inputs
- Password is hashed using bcrypt
- User is stored in the database
- Success response is returned

2.User Login

- POST /api/auth/login
- Steps:
- System verifies email and password
- JWT token is generated
- Token is sent in the response
- User becomes authenticated

3. Access Protected Routes

- User sends requests with:
- Authorization: Bearer <token>
- Middleware verifies the token.
- If valid → Request proceeds
- If invalid → Access denied

4.Book Operations

Authenticated users can perform:

- ➕ Add Book → POST `/api/book/add`
- 📖 Get Books → GET `/api/book`
- ✏️ Update Book → PUT `/api/book/:id`
- ❌ Delete Book → DELETE `/api/book/:id`

Each book is linked to the logged-in user.

5.Logout

- POST /api/auth/logout
- Token is stored in blacklist
- User session becomes invalid

6.Delete User(Optional)

- DELETE request removes the user account from the database.

Overall Flow Summary

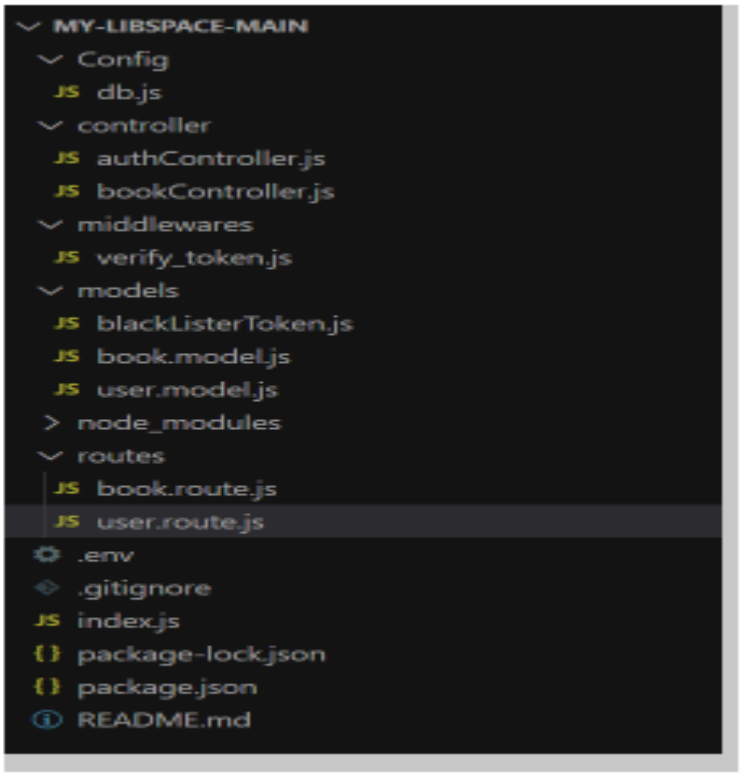
Register → Login → Receive Token → Access Protected Routes → Perform CRUD Operations → Logout

```
Authorization: Bearer <token>
```

Project Setup and Configuration:

Folder Structure:

```
MY-LIBSPACE-MAIN/
│
├── Config/
│   └── db.js
│
├── controller/
│   ├── authController.js
│   └── bookController.js
│
├── middlewares/
│   └── verify_token.js
│
├── models/
│   ├── blackListerToken.js
│   ├── book.model.js
│   └── user.model.js
│
├── routes/
│   ├── book.route.js
│   └── user.route.js
│
├── node_modules/
│
├── .gitignore
├── index.js
├── package.json
├── package-lock.json
└── README.md
```



Main Server File

File: index.js

This file is the main entry point of the LibSpace application.
It initializes the Express server, connects to the database, and loads the application routes.

```
import express from "express";
import dotenv from "dotenv";
import connectDb from "./Config/db.js";
import userRouter from "./routes/user.route.js";
import bookRouter from "./routes/book.route.js";
dotenv.config();

const app = express();

app.use(express.json());

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  connectDb();
  console.log(`Our server is working at PORT :${PORT}`);
});

app.use("/api/authuser", userRouter);
app.use("/api/book", bookRouter);
```

Figure: Main Server Setup (index.js)

Project Setup:

1. Install Dependencies

Install all required npm packages:

```
npm install
```

2. Configure Environment Variables

Create .env file:

```
PORT=5000

MONGO_URI=your_database_connection_string

JWT_SECRET=your_secret_key
```

PORT=5000
MONGO_URI=your_database_connection_string
JWT_SECRET=your_secret_key

3.Start Database

- Make sure MongoDB/MySQL is installed and running on your system.
- Verify the connection string matches your database configuration.

4. Run Server

Start the application using:

```
npm start
```

Or (if using nodemon):

```
npm run dev
```

Access Application

Open your browser and navigate to:

http://localhost:5000

The LibSpace application should now be running successfully.

Backend Development

The backend of LibSpace is developed using **Node.js and Express.js**, following RESTful architecture principles. It handles authentication, business logic, database operations, and API responses. The backend ensures secure data handling, structured routing, and efficient communication with MongoDB.

1.API Development & Routing

The backend APIs are built using Express.js. Routes are structured into separate files for user and book operations to maintain modularity.

- User Routes handle:
    - Registration
    - Login
    - Logout
    - Delete User

```
import userController from "../controller/authController.js";
import express from "express";
import { verifyToken } from "../middlewares/verify_token.js";
let userRouter = express.Router();




userRouter.post("/register", userController.registerUser);
userRouter.post("/login", userController.loginUser);
userRouter.post("/logout", userController.logoutUser);

userRouter.delete("/:userId", verifyToken, userController.deleteUser);

export default userRouter;
```

- Book Routes handle:
    - Add Book
    - Get Book
    - Update Book
    - Delete Book

```
import express from "express";
import { verifyToken } from "../middlewares/verify_token.js";
import bookController from "../controller/bookController.js";

const bookRouter = express.Router();


bookRouter.post("/addbook", verifyToken, bookController.addBook);
bookRouter.get("/getBooks", verifyToken, bookController.getAllBooks);
//  authT,
bookRouter.get("/:bookId", verifyToken, bookController.getBookById);
bookRouter.put("/:bookId", verifyToken, bookController.updateBook);
bookRouter.delete(
  "/deleteBook/:bookId",
  verifyToken,
  bookController.deleteBook,
);



export default bookRouter
```

Each route forwards requests to corresponding controllers where business logic is implemented.

2. **Database Integration & Authentication**

MongoDB is used as the database, integrated using Mongoose for schema modeling and data management.

- User passwords are securely hashed using bcrypt.
- JWT (JSON Web Token) is generated during login for authentication.
- Protected routes use middleware to verify tokens.
- A blacklist collection is maintained to invalidate tokens during logout.

This ensures secure access control and efficient data management within the backend system.

DATABASE DEVELOPMENT:

The database for LibSpace is developed using MongoDB, a NoSQL document-based database.

The application stores two main collections:

1. Users
2. Books

A one-to-many relationship is maintained:

- One User → Can have multiple Books
- Each Book → Belongs to one User (via ObjectId reference)

MongoDB stores data in JSON-like documents, making it flexible and scalable.

CONFIGURE MongoDB:

MongoDB is configured using Mongoose, an ODM (Object Data Modeling) library for Node.js.

Steps:

- Install mongoose:

```
npm install mongoose
```

- Create a database connection file inside Config/db.js
- Connect using MongoDB URI

Environment variable (.env):

```
MONGO_URI=mongodb://127.0.0.1:27017/libspace
```

CREATE DATABASE CONNECTION

File: Config/db.js

```
import mongoose from "mongoose";

const connectDb = () => {
  mongoose
    .connect(process.env.MONGO_URI)
    .then((res) => {
      console.log("Connected to mongodb");
    })
    .catch((err) => {
      console.log("Error while connection", err);
    });
};
export default connectDb;
```

CREATE SCHEMA AND MODELS:

Schemas define the structure of documents inside MongoDB collections.

- **User Schema**

File: models/user.model.js

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
    },
    email: {
      type: String,
      required: true,
    },
    password: {
      type: String,
      required: true,
    },
  },
  { versionKey: false, timestamps: true },
);

const User = mongoose.model("User", userSchema);

export default User;
```

```
const mongoose = require("mongoose");


const userSchema = new mongoose.Schema(

  {

    name: { type: String, required: true },
```

```
    email: { type: String, required: true },

    password: { type: String, required: true },

  },

  { versionKey: false, timestamps: true }

);


const UserModel = mongoose.model("User", userSchema);


module.exports = UserModel;
```

- **Book Schema**

File: models/user.model.js

```
import mongoose from "mongoose";

const bookSchema = new mongoose.Schema({
  url: {
    type: String,
    required: true,
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  title: {
    type: String,
    required: true,
  },
  author: {
    type: String,
    required: true,
  },
});
let Book = mongoose.model("Book", bookSchema);

export default Book;
```

File: models/book.model.js

```
const mongoose = require("mongoose");


const bookSchema = new mongoose.Schema({

  url: { type: String, required: true },

  user: {

    type: mongoose.Schema.Types.ObjectId,

    ref: "User",

    required: true,

  },

  title: { type: String, required: true },
```

```
  author: { type: String, required: true },

});



const BookModel = mongoose.model("Book", bookSchema);



module.exports = BookModel;
```

Token Blacklist Schema

File: models/blacklistToken.js

```
import mongoose from "mongoose";

const blacklistSchema = new mongoose.Schema({
  token:{
    type:String
  }
})

const BlackList = mongoose.model('BlackList',blacklistSchema);

export default BlackList
```

Figure: Token Blacklist Schema for storing logged-out JWT tokens

Authentication Middleware

File: middlewares/verifyToken.js

This middleware verifies the JWT token sent in the request header and ensures that only authenticated users can access protected routes.

```
import jwt from "jsonwebtoken";
import BlackList from "../models/blackListerToken.js";

export const verifyToken = async (req, res, next) => {
  let token = req.headers.authorization.split(" ")[1];
  let findBlock = await BlackList.findOne({ token: token });
  if (findBlock) {
    return res.status(200).send({ msg: "You are already logout" });
  }
  jwt.verify(token, process.env.SECRET_KEY, (err, decoded) => {
    if (err) {
      return res
        .status(401)
        .send({ msg: "You're not authenticated person", Error: err.message });
    }

    console.log(decoded);
    req.user = decoded;
    next();
  });
};
```

Project Execution

Project execution steps:

1.Install Dependencies

Open terminal inside the project folder and run:

```
npm install
```

This installs all required packages from package.json

2.Start Server

Run the following command:

npm start

OR (if using nodemon):

npm run dev

3. Database Connection

When the server starts:

- connectDb() connects to MongoDB.
- If successful, the console displays:

Connected to Database

Our server is working at PORT :5000

.Test API Endpoints

Use Postman or any API testing tool to test the endpoints:

User Routes

- POST → /api/auth/register
- POST → /api/auth/login
- DELETE → /api/auth/delete

Book Routes

- POST → /api/book/add
- GET → /api/book/
- PUT → /api/book/:id
- DELETE→ /api/ book/:id

Base URL:

- http://localhost:5000

Output Screenshots:

Output on terminal once server starts:

```
> my-libspace-main@1.0.0 dev
  > node index.js

  Connected to Database

 [9:13:26 PM] Our server is working at PORT :5000

> @ imeJs/js: node

node ✓
```

User registration output:

libspace / **userRegister**

POST http://localhost:5000/api/authuser/register

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "name":"Sri Chandana",
3   "email": "chandana3@gmail.com",
4   "password": "123456"
5
6 }
```

Body Cookies Headers (7) Test Results 200 OK · 851 ms · 293 B

{} JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "User registered successfully!"
4 }
```

- **User Login output:**

POST http://localhost:5000/api/authuser/login

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON


```
{
    "email": "chandana3@gmail.com",
    "password": "123456"
}
```


Body Cookies Headers (7) Test Results 200 OK 209 ms 739 B

{} JSON Preview Visualize


```
{
    "msg": "Logged In Successfully",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJpZCI6IjY5OTZiMDBhODZmMWE3MzQyYjlkMDYwNSIsImVtYWlsIjoiY2hhbmRhbmEzQGdtYWlsLmN
        3MTQ4MzQxMiwiZXhwIjoxNzcxNDg3MDEyfQ.MUTwlsM7iGGeZyEZmU3chsMBfOJhLHlpKCaccdRTZQs
    "user": [
        {
            "_id": "6996b00a86f1a7342b9d0605",
            "name": "Sri Chandana",
            "email": "chandana3@gmail.com",
            "password": "$2b$10$90BW1X/3Q/.kqmjyFCLAoOq/Q2Z9RHg6fCQVhMZf80..zSwkVDydm",
            "createdAt": "2026-02-19T06:39:07.014Z",
```


- **Adding Books:**

libspace / **addBook**

POST http://localhost:5000/api/book/addbook

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON


```
{
    "url":"www.libspacebook.com",
    "title": "think like a monk",
    "author": "jay shetty"
}
```


Body Cookies Headers (7) Test Results 200 OK 83 ms 268 B

{} JSON Preview Visualize


```
{
    "msg": "Book added successfully"
}
```

- **Get all Books:**

libspace / **getallBooks**

GET http://localhost:5000/api/book/getBooks

Docs Params Authorization Headers (7) Body Scripts Settings

Auth Type

Be...

Token

The authorization header will be automatically generated when you send the request. Learn more

Body Cookies Headers (7) Test Results 200 OK 38 ms 393 B

JSON Preview Visualize


```
[
    {
        "_id": "6996b28486f1a7342b9d060c",
        "url": "www.libspacebook.com",
        "user": "6996b00a86f1a7342b9d0605",
        "title": "think like a monk",
        "author": "jay shetty",
        "__v": 0
    }
]
```


- **Get book by ID:**

libspace / **updateBook**

PUT http://localhost:5000/api/book/6996b28486f1a7342b9d060c

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

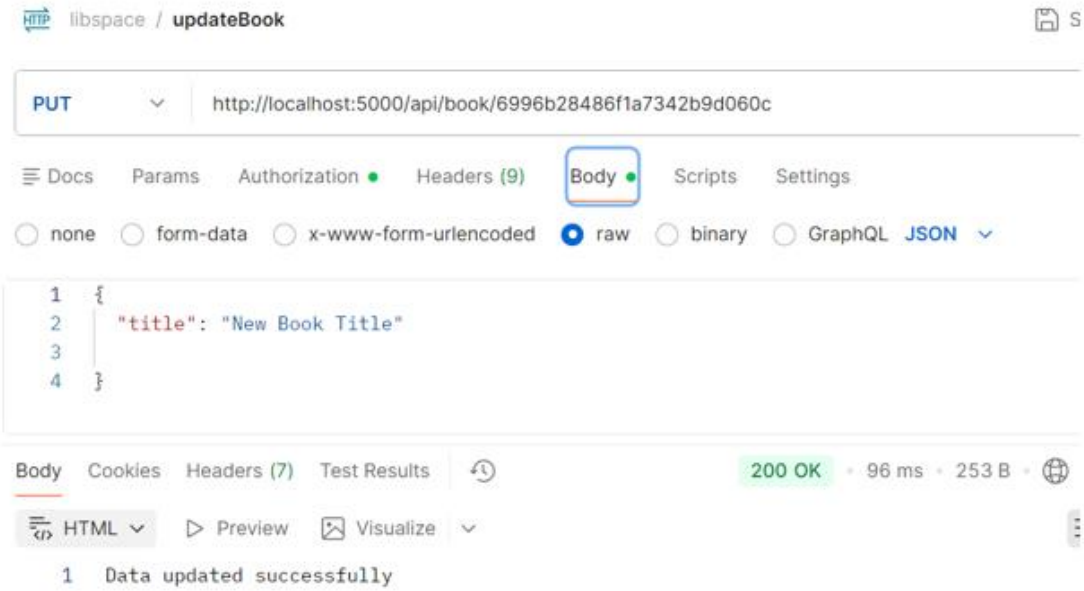

```
{
  "title": "New Book Title"

}
```


Body Cookies Headers (7) Test Results 200 OK 96 ms 253 B

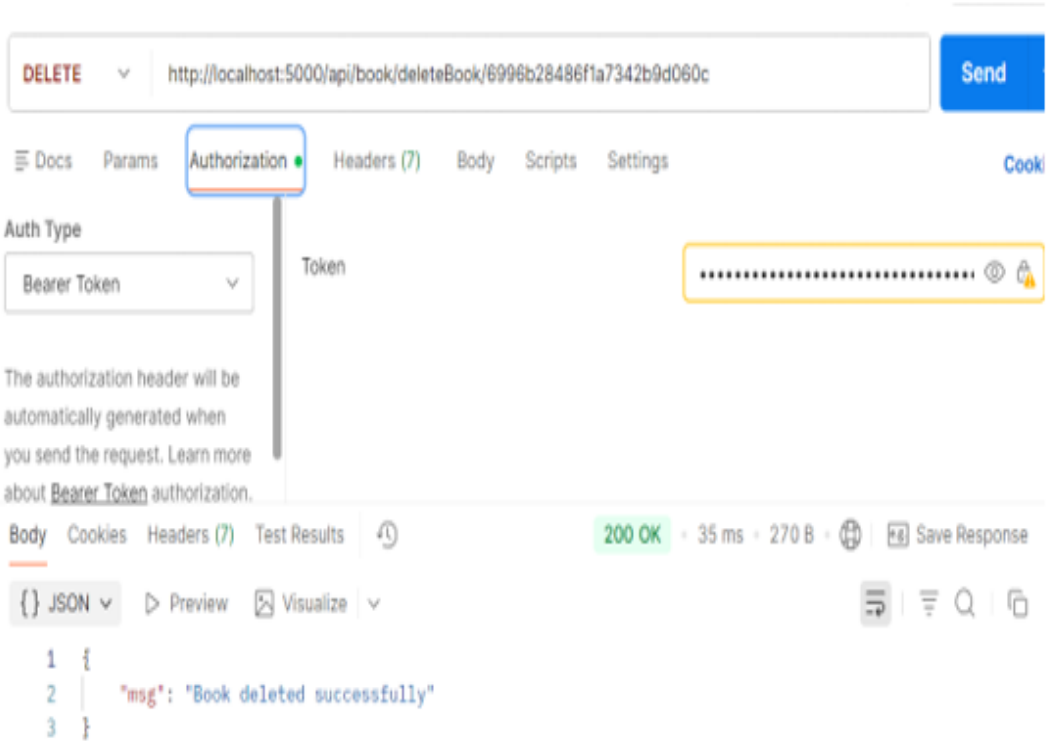
HTML Preview Visualize

```
Data updated successfully
```

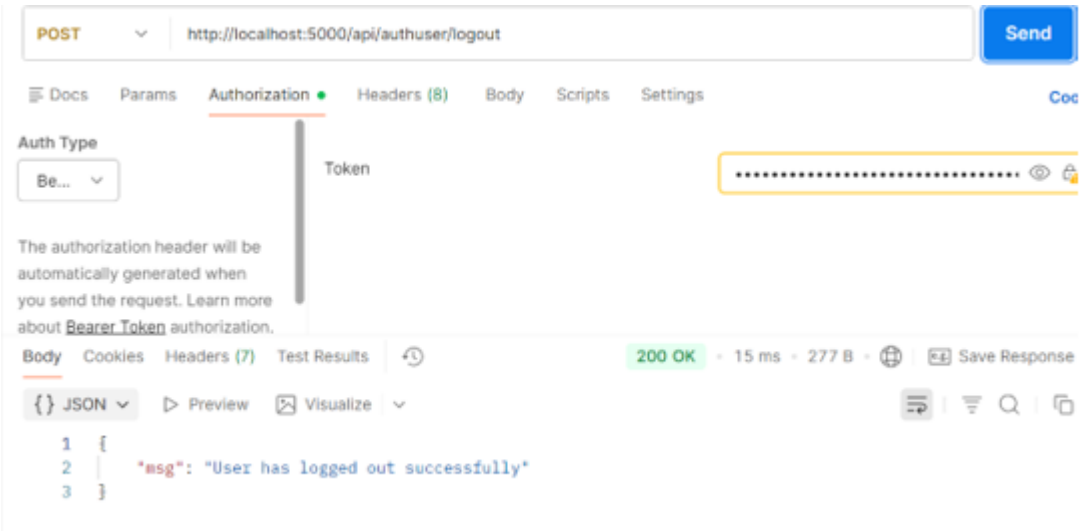
- **Update book:**



- **Delete book:**



- **User logout:**



Demo video:

LibSpaceDemo

Drive Link : Libspace code